

Q: What are the four pillars of OOPs?

Answer: Encapsulation, Abstraction, Inheritance, and Polymorphism.

Q: Explain Encapsulation in simple terms?

Answer: It is the wrapping of data (variables) and code (methods) into a single unit (class), hiding internal state using private modifiers and exposing it via public getters/setters.

Q: What is the difference between Abstraction and Encapsulation?

Answer: Abstraction hides the internal implementation complexities (what the object does), whereas Encapsulation hides the internal data state (how it does it).

Q: Explain Compile-time vs. Run-time Polymorphism?

Answer: Compile-time is achieved through Method Overloading (same method name, different parameters). Run-time is achieved through Method Overriding (subclass provides a specific implementation of a parent's method).

Q: Why doesn't Java support multiple inheritance with classes?

Answer: To avoid the "Diamond Problem" where a class inherits conflicting implementations of the same method from multiple parent classes.

Q: What is the difference between final, finally, and finalize?

Answer: final is a keyword to restrict modification (constant variables, un-overridable methods, un-inheritable classes). finally is a block used in exception handling to execute crucial code regardless of an exception. finalize() is a deprecated object method called by the Garbage Collector before memory cleanup.

Q: Can we override a static method?

Answer: No. Static methods belong to the class, not the instance. If you declare a static method with the same signature in a subclass, it's called Method Hiding, not overriding.

Q: Can you overload the main method?

Answer: Yes, you can have multiple main methods, but the JVM will only use the standard public static void main(String[] args) as the entry point.

Q: Can you override a private method?

Answer: No. Private methods are not visible to subclasses, so they are bound at compile-time (Static Binding).

Q: Abstract Class vs. Interface (Java 8+)?

Answer: Abstract classes can have state (instance variables) and constructors. Interfaces cannot hold state (variables are implicitly public static final) but can have default and static methods.

Q: Can an abstract class be instantiated?

Answer: No, you cannot use the new keyword on an abstract class, but it can have constructors that are called when a subclass is instantiated.

Q: Can an interface implement another interface?

Answer: No, an interface extends another interface. Classes implement interfaces.

Q: What happens if a class implements two interfaces that have the same default method?

Answer: The compiler will throw an error. You must explicitly override the method in the implementing class to resolve the ambiguity.

Q: What are Access Modifiers and their scopes?

Answer: private (class level), default (package level), protected (package + subclass level), and public (global).

Q: What is the super keyword?

Answer: It acts as a reference variable used to refer to immediate parent class objects, invoke parent constructors, or call parent methods.

Q: What is the this keyword?

Answer: It refers to the current object instance within a method or constructor.

Q: Can a constructor be static, final, or abstract?

Answer: No. Constructors are meant to initialize objects, not belong to the class (static), they aren't inherited (final), and they must have a body (abstract).

Q: Does Java support Pass-by-Value or Pass-by-Reference?

Answer: Java is strictly Pass-by-Value. When passing an object, the reference to the object is passed by value (a copy of the memory address).

Q: Why is String immutable in Java?

Answer: For security, thread-safety, memory optimization (String Pool), and to allow Strings to be used safely as keys in HashMaps.

Q: String vs. StringBuilder vs. StringBuffer?

Answer: String is immutable. StringBuilder is mutable and not thread-safe (faster). StringBuffer is mutable and thread-safe (slower due to

synchronized methods).

Q: What is the String Constant Pool?

Answer: A special memory area in the Heap where the JVM stores unique literal Strings to save memory space.

Q: What does the intern() method do?

Answer: It explicitly checks if a String exists in the String Pool. If it does, it returns the reference; if not, it adds it to the pool.

Q: Difference between == and .equals()?

Answer: == compares memory addresses (reference equality). .equals() compares the actual content/value of the objects.

Q: If String s1 = new String("A") and String s2 = new String("A"), what is s1 == s2?

Answer: false. The new keyword forces the creation of distinct objects in the heap memory, ignoring the String Pool.

Q: What is a Wrapper class?

Answer: Classes like Integer, Boolean, and Double that convert primitive data types into objects so they can be used in Collections.

Q: What is Autoboxing and Unboxing?

Answer: Autoboxing is automatically converting a primitive to its wrapper class (e.g., int to Integer). Unboxing is the reverse.

Q: What is a Marker Interface?

Answer: An interface with no methods or fields (e.g., Serializable, Cloneable). It signals to the JVM that the class possesses a special property.

Q: Shallow Copy vs. Deep Copy?

Answer: Shallow copy creates a new object but inserts references to the original nested objects. Deep copy creates a completely independent copy of the object and all its nested objects.

Q: What is a Singleton class?

Answer: A class that allows only one instance of itself to be created per JVM.

Q: How do you create a Singleton class?

Answer: Make the constructor private, declare a private static instance of the class, and provide a public static getter method to return that instance.

Q: How can you break a Singleton?

Answer: Through Reflection, Serialization, or Object Cloning. (Using Enum singletons prevents this).

Q: What is an inner class?

Answer: A class declared inside another class, used to logically group classes and increase encapsulation.

Q: Difference between break and continue?

Answer: break completely exits the current loop. continue skips the current iteration and jumps to the next one.

Q: What is Variable Shadowing?

Answer: When a local variable has the same name as an instance variable, the local variable "shadows" (hides) the instance variable in that scope.

Q: What is the instanceof keyword?

Answer: An operator used to test whether an object is an instance of a specific class, subclass, or interface.

Q: How does HashMap work internally?

Answer: It uses an array of Nodes (buckets). When put(K, V) is called, it calculates the hash of the key to find the bucket index. If a collision occurs (same index), it stores the entry in a LinkedList. In Java 8+, if the LinkedList length exceeds 8, it dynamically transforms into a Red-Black Tree to optimize search time.

Q: Explain the contract between hashCode() and equals()?

Answer: If two objects are equal according to equals(), they must return the same hashCode(). However, two objects with the same hashCode() do not necessarily have to be equals().

Q: What happens if you override equals() but not hashCode() in a Map key?

Answer: The HashMap will place identical objects in different buckets. You won't be able to retrieve the value because the Map looks for the bucket using the hashcode first.

Q: ArrayList vs. LinkedList?

Answer: ArrayList uses a dynamic array—faster for retrieval $O(1)$. LinkedList uses a doubly-linked list—faster for insertions/deletions in the middle $O(1)$ once found.

Q: HashSet vs. TreeSet?

Answer: HashSet is unordered and backed by a HashMap. TreeSet is sorted in ascending natural order and backed by a TreeMap (Red-Black tree).

Q: How does HashSet internally work?

Answer: It uses a HashMap under the hood where your inserted value becomes the key, and a dummy constant object is used as the value.

Q: Comparable vs. Comparator?

Answer: Comparable is for default natural sorting and requires modifying the class to implement compareTo(). Comparator is for custom sorting logic and is implemented externally via compare().

Q: Hashtable vs. HashMap?

Answer: Hashtable is thread-safe, synchronized, and does not allow null keys/values. HashMap is not thread-safe, faster, and allows one null key and multiple null values.

Q: What is the Iterable interface?

Answer: The root interface for the Collection hierarchy, providing the iterator() method to sequentially traverse elements.

Q: Iterator vs. ListIterator?

Answer: Iterator traverses forward only. ListIterator (only for Lists) can traverse bidirectionally and add/modify elements.

Q: How do you avoid ConcurrentModificationException?

Answer: Instead of a for-each loop, use Iterator.remove() or switch to thread-safe collections like CopyOnWriteArrayList or ConcurrentHashMap.

Q: What is PriorityQueue?

Answer: A queue that orders elements based on their natural ordering or a supplied Comparator, rather than FIFO (First-In-First-Out).

Q: How does TreeMap work?

Answer: It stores key-value pairs sorted by keys using a Red-Black Tree implementation, ensuring $O(\log n)$ performance for basic operations.

Q: What is the difference between poll() and remove() in a Queue?

Answer: Both remove and return the head of the queue. However, if the queue is empty, poll() returns null, while remove() throws an exception.

Q: What is the difference between peek() and element()?

Answer: Both return the head without removing it. If empty, peek() returns null, and element() throws an exception.

Q: What is LinkedHashMap?

Answer: A subclass of HashMap that maintains a doubly-linked list running through its entries, preserving insertion order.

Q: Can you store null keys in TreeMap?

Answer: No, it will throw a NullPointerException because it must compare keys to sort them.

Q: How does Collections.sort() work internally?

Answer: It converts the collection to an array, uses Dual-Pivot Quicksort (for primitives) or Timsort (for objects), and writes the sorted array back to the collection.

Q: What is the Time Complexity of HashMap get() and put()?

Answer: Average case is $O(1)$. In the worst-case scenario (many collisions), it's $O(\log n)$ in Java 8+ due to Treeification.

Q: What is the Default Capacity and Load Factor of HashMap?

Answer: Default capacity is 16. The load factor is 0.75.

Q: What happens when HashMap exceeds its Load Factor?

Answer: Rehashing occurs. The capacity is doubled, and all existing entries are recalculated and moved to new buckets.

Q: What is a Hash Collision?

Answer: When two distinct keys compute the same index in the hash table array.

Q: What is the difference between Collection and Collections?

Answer: Collection is an interface representing a group of objects. Collections is a utility class containing static helper methods (like sorting and synchronizing).

Q: How do you make an ArrayList read-only?

Answer: By wrapping it using Collections.unmodifiableList(list).

Q: What is CopyOnWriteArrayList?

Answer: A thread-safe variant of ArrayList where all mutative operations (add, set) create a fresh copy of the underlying array, eliminating locks during reads.

Q: What is a BlockingQueue?

Answer: A queue that is thread-safe. It blocks the retrieving thread if the queue is empty, and blocks the inserting thread if the queue is full.

Q: Explain IdentityHashMap?

Answer: A map that uses reference equality (==) instead of object equality (equals()) to compare keys.

Q: What is the Vector class?

Answer: A legacy, thread-safe, synchronized dynamic array. Rarely used now because it is slow; ArrayList is preferred.

Q: How does Stack work in Collections?

Answer: It is a legacy class extending Vector, implementing LIFO (Last-In-First-Out) logic via push(), pop(), and peek().

Q: What is the difference between clear() and removeAll()?

Answer: clear() empties the collection completely in O(n) time. removeAll(Collection c) removes only the elements present in the specified collection.

Q: How do you create a thread in Java?

Answer: By either extending the Thread class or implementing the Runnable interface.

Q: Extend Thread vs. Implement Runnable?

Answer: Implementing Runnable is better because Java doesn't support multiple inheritance. If you extend Thread, your class cannot extend any other class.

Q: Difference between start() and run()?

Answer: start() creates a new underlying OS thread and then calls run(). If you call run() directly, it just executes synchronously in the current main thread like a normal method.

Q: What are the Thread states (Thread Lifecycle)?

Answer: New, Runnable, Blocked, Waiting, Timed_Waiting, and Terminated.

Q: What is Synchronization?

Answer: A mechanism to ensure that only one thread can access a shared resource or execute a specific block of code at a time.

Q: Object-level lock vs. Class-level lock?

Answer: Object-level locks (synchronized non-static method) prevent multiple threads from accessing the same instance. Class-level locks (synchronized static method) prevent multiple threads from accessing any instance of the class.

Q: What is a Race Condition?

Answer: A scenario where multiple threads read and write a shared variable simultaneously, leading to unpredictable, incorrect final values.

Q: What is a Deadlock?

Answer: When two or more threads are blocked forever, each waiting for a lock that the other thread is currently holding.

Q: How can you prevent Deadlocks?

Answer: By avoiding nested locks, ensuring threads acquire multiple locks in the exact same order globally, and using timeouts (`tryLock()`).

Q: What is the `volatile` keyword?

Answer: It guarantees visibility of variable changes across threads. It instructs the JVM to read the variable directly from main memory rather than CPU caches.

Q: `volatile` vs. `synchronized`?

Answer: `volatile` only ensures visibility, it does not guarantee atomicity (e.g., `count++` is still not thread-safe). `synchronized` guarantees both visibility and atomicity by locking the resource.

Q: Explain `wait()`, `notify()`, and `notifyAll()`?

Answer: Used for inter-thread communication. `wait()` pauses the thread and releases the lock. `notify()` wakes up one waiting thread. `notifyAll()` wakes up all waiting threads.

Q: Why must `wait()` be called inside a `synchronized` block?

Answer: To ensure the thread holds the monitor lock of the object before pausing. Otherwise, it throws `IllegalMonitorStateException`.

Q: `sleep()` vs. `wait()`?

Answer: `sleep()` pauses the thread for a specified time but keeps the lock. `wait()` pauses the thread indefinitely (until notified) and releases the lock.

Q: What is `join()`?

Answer: When `threadA.join()` is called within the main thread, the main thread waits until `threadA` finishes executing before continuing.

Q: What is `yield()`?

Answer: A hint to the thread scheduler that the current thread is willing to yield its current use of a processor to let other threads of equal priority run.

Q: What is a Daemon Thread?

Answer: A background service thread (like the Garbage Collector). The JVM will terminate itself if only Daemon threads are left running.

Q: Can you start a thread twice?

Answer: No. Calling `start()` on an already started thread throws an `IllegalThreadStateException`.

Q: What is `ThreadLocal`?

Answer: A class that provides variables that can only be read and written by the same thread, ensuring thread safety without synchronization.

Q: What are ReentrantLocks?

Answer: An explicit lock from `java.util.concurrent.locks` that allows the same thread to acquire the lock multiple times without deadlocking itself.

Q: ReentrantLock vs. synchronized?

Answer: ReentrantLock provides advanced features like fairness (longest waiting thread gets the lock next), ability to interrupt waiting threads, and `tryLock()` (don't block if lock is unavailable).

Q: What is a Semaphore?

Answer: A concurrency utility that restricts the number of threads accessing a specific resource using a predefined set of permits.

Q: What is a CountdownLatch?

Answer: A synchronizer that allows one or more threads to wait until a set of operations being performed by other threads completes (counts down to zero).

Q: What is a CyclicBarrier?

Answer: Similar to CountdownLatch, but it allows a set of threads to wait for each other to reach a common barrier point before proceeding.

Q: How does ConcurrentHashMap work?

Answer: It avoids locking the entire map. In Java 8+, it uses fine-grained locking at the bucket level (Node) using CAS (Compare-And-Swap) operations and synchronized blocks.

Q: Callable vs. Runnable?

Answer: Both represent tasks. `Runnable.run()` cannot return a result or throw checked exceptions. `Callable.call()` can return a result (via `Future`) and throw exceptions.

Q: What is the Future interface?

Answer: It represents the result of an asynchronous computation, providing methods to check if the computation is complete, wait for its completion, and retrieve the result.

Q: What is the ExecutorService?

Answer: A higher-level API for managing thread pools, saving you from manually creating and destroying Thread instances.

Q: What types of Thread Pools does Executors provide?

Answer: `FixedThreadPool`, `CachedThreadPool`, `SingleThreadExecutor`, and `ScheduledThreadPool`.

Q: `shutdown()` vs. `shutdownNow()`?

Answer: `shutdown()` waits for currently executing and queued tasks to finish. `shutdownNow()` attempts to forcefully stop executing tasks and discards queued ones.

Q: What is `AtomicInteger`?

Answer: A class that provides thread-safe operations (like `incrementAndGet()`) on an integer without needing synchronized, using CAS underneath.

Q: Explain CAS (Compare-And-Swap)?

Answer: A lock-free atomic instruction. It compares a memory location's current value with an expected value, and if they match, updates it to a new value.

Q: What is Thread Starvation?

Answer: When low-priority threads are perpetually denied access to CPU time because high-priority threads are hogging it.

Q: What is Livelock?

Answer: When threads are continually changing state in response to each other, so neither makes progress, but they aren't completely blocked like in a Deadlock.

Q: How do you gracefully stop a thread?

Answer: By using a volatile boolean flag to check inside the `run()` loop, or by using `Thread.interrupt()` and checking `Thread.currentThread().isInterrupted()`.

Q: What are the key features introduced in Java 8?

Answer: Lambda expressions, Streams API, Functional Interfaces, Default/Static methods in Interfaces, Optional, and the new Date/Time API.

Q: Which Java version do you use and why?

Answer: I use Java 8 for core logic since it introduced the most paradigms, but utilize LTS versions like Java 11 or 17 for better performance, pattern matching, and record classes.

Q: What is a Lambda Expression?

Answer: An anonymous function (without a name or modifier) that provides a clean, concise way to implement a Functional Interface.

Q: What is a Functional Interface?

Answer: An interface that contains exactly one abstract method. It can have multiple default or static methods. Annotated with `@FunctionalInterface`.

Q: Name four built-in Functional Interfaces?

Answer: Predicate (returns boolean), Function (takes T, returns R), Consumer (takes T, returns void), Supplier (takes nothing, returns T).

Q: What is the Streams API?

Answer: A functional approach to processing collections of objects in a declarative manner (filtering, mapping, reducing) without modifying the underlying data.

Q: Intermediate vs. Terminal Stream operations?

Answer: Intermediate operations (like `filter()`, `map()`) return a new Stream and are lazy. Terminal operations (like `collect()`, `forEach()`) produce a result or side-effect and trigger the Stream execution.

Q: `map()` vs. `flatMap()`?

Answer: `map()` is used for one-to-one transformation. `flatMap()` is used for one-to-many transformations, flattening nested arrays/streams into a single continuous stream.

Q: What is the Optional class?

Answer: A container object that may or may not contain a non-null value, forcing the programmer to handle the absence of a value properly without throwing `NullPointerException`.

Q: How do you create an Optional?

Answer: `Optional.of(value)` (throws exception if null), `Optional.ofNullable(value)` (safe), `Optional.empty()`.

Q: `Optional.get()` vs. `Optional.orElse()`?

Answer: `get()` throws an exception if the value is absent. `orElse()` safely returns the value if present, or a fallback default value if not.

Q: Why were default methods added to interfaces?

Answer: To allow adding new methods to interfaces without breaking existing implementing classes (backward compatibility).

Q: Can an interface have static methods in Java 8?

Answer: Yes, they are used to define utility/helper methods directly within the interface.

Q: What are Method References?

Answer: Shorthand syntax `::` to refer to an existing method instead of

writing a lambda expression (e.g., `System.out::println`).

Q: How does `Stream.filter()` work?

Answer: It takes a Predicate and returns a stream containing only the elements that match the given condition.

Q: How does `Stream.reduce()` work?

Answer: It takes a stream of elements and combines them into a single result (e.g., summing all numbers in a list).

Q: What is the Date-Time API?

Answer: The `java.time` package (e.g., `LocalDate`, `ZonedDateTime`) introduced to replace the deeply flawed, mutable, and thread-unsafe `java.util.Date`.

Q: Parallel Streams vs. Sequential Streams?

Answer: Sequential executes iteratively on one thread. Parallel Stream divides the tasks into subtasks running concurrently on multiple cores using the ForkJoin pool.

Q: When should you avoid Parallel Streams?

Answer: When processing tiny collections, when order strictly matters, or when the overhead of splitting the data is larger than the parallel execution benefit.

Q: What is `CompletableFuture`?

Answer: An extension of the Future API that allows you to chain multiple asynchronous tasks together, handle exceptions natively, and write non-blocking asynchronous code.

Q: How do you find the maximum number in a list using Streams?

Answer: `list.stream().max(Integer::compareTo).get();`

Q: How do you count duplicates in a List using Streams?

Answer: By using `Collectors.groupingBy(Function.identity(), Collectors.counting())`.

Q: What is `StringJoiner`?

Answer: A utility class used to construct a sequence of characters separated by a delimiter (e.g., joining CSV strings).

Q: What is Nashorn in Java 8?

Answer: A lightweight JavaScript engine that allowed executing JS code inside the JVM (deprecated and removed in newer Java versions).

Q: What is Metaspace?

Answer: In Java 8, PermGen memory was completely removed and replaced

by Metaspace, which dynamically resizes and utilizes native OS memory to store class metadata.

Q: What is the Java Exception hierarchy?

Answer: Throwable is the root class. It has two main subclasses: Error (severe system problems) and Exception (recoverable issues).

Q: Checked vs. Unchecked Exceptions?

Answer: Checked exceptions (like IOException) are checked at compile-time and must be handled. Unchecked exceptions (RuntimeException like NullPointerException) occur at run-time due to logical errors.

Q: throw vs. throws?

Answer: throw is used inside a method body to explicitly throw an exception object. throws is used in the method signature to declare that the method might throw exceptions.

Q: Can we have a try block without a catch block?

Answer: Yes, but only if it is immediately followed by a finally block or if it's a try-with-resources statement.

Q: Does the finally block always execute?

Answer: Almost always. The only exceptions are if System.exit(0) is called, the JVM crashes, or the thread executing it is killed.

Q: If both try and finally return a value, which is returned?

Answer: The value returned from the finally block completely overrides the return value from the try block.

Q: What is Try-With-Resources?

Answer: A syntax try (Resource r = new Resource()) that automatically closes resources (files, streams, database connections) when the try block finishes, preventing memory leaks.

Q: How do you create a Custom Exception?

Answer: Create a class that extends Exception (for a checked exception) or RuntimeException (for an unchecked exception).

Q: Should you catch an Error in Java?

Answer: No. Errors like OutOfMemoryError indicate severe, unrecoverable JVM problems. Application code shouldn't try to handle them.

Q: NoClassDefFoundError vs. ClassNotFoundException?

Answer: NoClassDefFoundError occurs if a class was present during compile-time but is missing at runtime. ClassNotFoundException occurs when Class.forName() fails to dynamically load a class.

Q: How to handle NullPointerException elegantly?

Answer: By performing null checks, utilizing Optional, or adding default fallbacks.

Q: What happens if an exception is thrown inside a finally block?

Answer: It will mask any original exception thrown in the try or catch block, making debugging extremely difficult unless properly caught inside the finally block.

Q: Explain JVM architecture briefly?

Answer: It consists of a Class Loader Subsystem (loads, links, initializes classes), Runtime Data Areas (Memory), and an Execution Engine (Interpreter, JIT Compiler, Garbage Collector).

Q: Heap vs. Stack Memory in Java?

Answer: Heap space is used for dynamic memory allocation of Java objects and JRE classes at runtime (globally accessible). Stack memory is used for temporary storage of method calls, local variables, and primitive types (thread-safe, local).

Q: Where is the String Pool located?

Answer: Prior to Java 7, it was in the PermGen. From Java 7 onwards, it was moved directly into the main Heap memory.

Q: What causes OutOfMemoryError: Java heap space?

Answer: When the application creates too many objects and the Garbage Collector cannot free up enough space, usually due to a memory leak or a massive data processing spike.

Q: What causes StackOverflowError?

Answer: When the stack memory is completely exhausted, virtually always caused by infinite or deeply nested recursive method calls.

Q: How does Java Garbage Collection (GC) work?

Answer: It uses a "Mark-and-Sweep" mechanism. The GC scans the heap, marks all objects still referenced by the application as "alive", and sweeps (deletes) the unreferenced ones.

Q: Name some types of Garbage Collectors in Java?

Answer: Serial GC, Parallel GC (default in Java 8), G1 GC (default in Java 9+), and ZGC (low latency).

Q: What is the G1 Garbage Collector?

Answer: Garbage-First (G1) partitions the heap into smaller, equal-sized regions and collects garbage from the regions with the most dead objects first, ensuring predictable pause times.

Q: Explain Minor GC vs. Major GC?

Answer: Minor GC cleans up the "Young Generation" space. Major (Full) GC cleans up the "Old Generation" space. Major GC takes much longer and causes "Stop the World" pauses.

Q: What are the memory generations in the Heap?

Answer: Young Generation (divided into Eden Space, Survivor 0, and Survivor 1) and Old (Tenured) Generation. Objects that survive multiple Minor GCs get promoted to the Old Generation.

Q: What is a Memory Leak in Java?

Answer: Even with Garbage Collection, memory leaks occur when an application holds strong references to unused objects (e.g., adding objects to an unmanaged ArrayList), preventing the GC from cleaning them up.

Q: How do you detect and fix memory leaks?

Answer: By using profilers like VisualVM or JConsole to capture Heap Dumps, inspecting which objects are taking up space, and nullifying old references or using WeakHashMap.

Q: What is the finalize() method and why is it deprecated?

Answer: A method called by the GC right before destroying an object. It was deprecated in Java 9 because its execution timing is entirely unpredictable and it drastically slows down garbage collection.